TECHRISE 3.0

Cybersecurity & DevSecOps Programme · Abia Cohort 3.0 · 2026

PHASE 3 · DEVSECOPS & CLOUD SECURITY

WEEK 11

Container Security and Infrastructure as Code

Student Training Manual — AI-First Curriculum

PROGRAMME	COHORT	WEEK	PHASE	METHODOLOGY
Cybersecurity & DevSecOps	**Abia Cohort 3.0**	**11 of 12**	**3 — DevSecOps & Cloud Security**	**AI-First (Manual → AI Enhancement)**

Prepared by the TechRise Curriculum Design Team

Lead: Engr. Eduwem Adam · Co-facilitator: Mr. Akorita-Ifeanyichukwu Nehemiah

TABLE OF CONTENTS

- **Week 11: Container Security and Infrastructure as Code**
  - Phase: DevSecOps and Cloud Security (Weeks 9–12)
  - Day 1 — Containers vs Virtual Machines and Docker Security
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 2 — Container Vulnerability Scanning with Trivy
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 3 — Infrastructure as Code (IaC) and Terraform Security
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 4 — IaC Scanning with Checkov and Cloud Misconfiguration Detection
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 5 — Review, Assessment & Portfolio
    - LEARN · PRACTICE · BUILD · DOCUMENT
- **Weekly Checklist**
- **Skills Verification**
- **Self-Assessment Quiz**
- **Additional Resources**
- **Looking Ahead — Week 12**

Phase: DevSecOps and Cloud Security (Weeks 9–12) **Week 11 of 12**

WEEK 11: CONTAINER SECURITY AND INFRASTRUCTURE AS CODE

Weekly Overview

This week you will learn to secure the two foundational technologies of modern cloud infrastructure: containers and Infrastructure as Code (IaC). You will understand the difference between containers and virtual machines, master Docker security best practices, scan container images for vulnerabilities using Trivy, and learn to detect misconfigurations in Terraform code using Checkov. You will also explore the cloud shared responsibility model and IAM least privilege principles. By the end of this week, you will be able to assess container security posture and prevent cloud misconfigurations before they reach production.

How this week builds on Week 10

- Week 10 introduced CI/CD pipelines. This week you will add container scanning to those pipelines. Vulnerable base images will be caught automatically before deployment.

How this week sets up Week 12

- Week 12 is the Capstone project, where you will demonstrate a complete DevSecOps workflow including container security and IaC scanning.

Why this matters in a real security job

- Container misconfigurations and vulnerable base images are among the top 10 cloud breaches every year. The 2023 Toyota data breach exposed 2 million vehicles' data due to a misconfigured database container.
- Infrastructure as Code misconfigurations (public S3 buckets, open security groups) cause 70% of cloud data breaches. Securing containers and IaC is not optional — it is mandatory.

■ **DevSecOps Week Note: Container scanning and IaC scanning ARE the lesson. Every line of Dockerfile and Terraform must be explained with security context.**

DAY 1

Containers vs Virtual Machines and Docker Security

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Explain the difference between containers and virtual machines

2. Describe Docker architecture (images, containers, registries)

3. Identify common container security risks and mitigations

A CONTAINER

is a lightweight, standalone, executable package that includes everything needed to run software (code, runtime, system tools, libraries), sharing the host OS kernel for efficiency.

Virtual Machines vs Containers Architecture

```
■■■
```

Vulnerable base image	Using an outdated OS image with known CVEs	FROM ubuntu:18.04 (has 500+ CVEs)	Use minimal/distroless images; scan with Trivy
Running as root	Container process runs as UID 0	No USER directive in Dockerfile	Add USER nonroot in Dockerfile
Exposed secrets	Credentials baked into image layers	ENV DB_PASSWORD=secret in Dockerfile	Use Docker secrets or environment injection
Privileged mode	Container has full host access	docker run --privileged	Never use --privileged; use specific capabilities
Unrestricted network	Container on host network	docker run --network=host	Use custom bridge networks; restrict ports
No read-only filesystem	Container can write anywhere	Malware persistence after breach	Use --read-only flag; mount specific writable volumes
Outdated packages	OS packages inside image have CVEs	apt packages not updated at build time	Update during build; rescan regularly

Secure Dockerfile — Best Practices

DOCKERFILE — SECURE vs INSECURE

```
# INSECURE Dockerfile (many issues) FROM ubuntu:18.04 # Old, vulnerable base image RUN apt-get install python3 -y COPY app.py /app/ ENV DB_PASSWORD=supersecret # Secret baked in! CMD ['python3', '/app/app.py'] # Runs as root! # SECURE Dockerfile (best practices applied) FROM python:3.11-slim # Minimal, updated base image # Create non-root user RUN groupadd -r appuser && useradd -r -g appuser appuser WORKDIR /app # Install dependencies first (better layer caching) COPY requirements.txt . RUN pip install --no-cache-dir -r requirements.txt # Copy application code COPY --chown=appuser:appuser app.py . # Switch to non-root user USER appuser # No secrets — use environment variables at runtime # DB_PASSWORD is injected by orchestrator, not baked in EXPOSE 8080 CMD ['python3', 'app.py']
```

WHY THIS MATTERS

The 2023 Toyota Connected Services breach exposed data of 2.15 million customers over 10 years. A database container had been accidentally configured with public access. A Trivy scan of the container image would have flagged the misconfiguration. Regular container scanning (which Toyota was not doing) would have caught it on the first deployment.

PRACTICE *(2 Hours)*

LAB 11.1.1 — Build a Vulnerable and Secure Docker Image

Objective: Create two Docker images — one with security issues, one hardened.

Prerequisites: Docker installed; Kali Linux

Step 1 — Create the vulnerable Dockerfile

BASH

```
mkdir -p ~/labs/week11 cat > ~/labs/week11/Dockerfile.vulnerable << 'EOF' FROM ubuntu:18.04 RUN
apt-get update && apt-get install -y python3 python3-pip COPY app.py /app/ ENV DB_PASSWORD=admin123
ENV API_KEY=sk-secret123 WORKDIR /app CMD ["python3", "app.py"] EOF # Create simple app cat >
~/labs/week11/app.py << 'EOF' import os print('App running as user:', os.getuid())
print('DB_PASSWORD:', os.environ.get('DB_PASSWORD', 'not set')) EOF # Build the vulnerable image cd
~/labs/week11 docker build -f Dockerfile.vulnerable -t vulnerable-app:v1 . docker images | grep
vulnerable-app
```

Step 2 — Run the vulnerable container and inspect

BASH

```
# Run and check what user it runs as docker run --rm vulnerable-app:v1 # Inspect the image for exposed
secrets docker history vulnerable-app:v1 docker inspect vulnerable-app:v1 | grep -A5 'Env' # PROBLEM:
You can see DB_PASSWORD=admin123 in clear text! # Anyone with docker inspect access can read your
secrets
```

Step 3 — Create the secure Dockerfile

BASH

```
cat > ~/labs/week11/Dockerfile.secure << 'EOF' FROM python:3.11-slim RUN groupadd -r appuser &&
useradd -r -g appuser appuser WORKDIR /app COPY --chown=appuser:appuser app_secure.py . USER appuser
EXPOSE 8080 CMD ["python3", "app_secure.py"] EOF cat > ~/labs/week11/app_secure.py << 'EOF' import os
print('App running as user:', os.getuid()) # Secrets are injected at runtime, not baked in db_pass =
os.environ.get('DB_PASSWORD', 'NOT SET') print('DB connection configured:', 'YES' if db_pass != 'NOT
SET' else 'NO') EOF docker build -f Dockerfile.secure -t secure-app:v1 . # Run — secrets injected at
runtime, not baked in docker run --rm -e DB_PASSWORD=runtimesecret secure-app:v1
```

■ ANALYSE THIS

Run docker inspect on both images. What is different in the Env section? Run docker run --rm vulnerable-app:v1 and note the user ID. Run docker run --rm secure-app:v1 and note the user ID. Which is safer and why does the user ID matter for security?

BUILD *(2 Hours)*

BUILD BRIEF

Create a Docker Security Hardening Checklist and apply it to a real Dockerfile.

Required checklist items (minimum 10):

- Base image: use minimal/distroless, pin specific version (FROM python:3.11-slim, not FROM python:latest)
- Non-root user: create and use non-root USER in every Dockerfile
- No secrets in image: verify docker inspect shows no passwords, keys, tokens
- Read-only filesystem: add --read-only to docker run where possible
- Port exposure: EXPOSE only required ports (not -p 0.0.0.0:80:80)
- No --privileged flag: document why it is never acceptable
- Health checks: add HEALTHCHECK instruction to Dockerfile

- Multi-stage builds: use multi-stage to reduce final image size and attack surface
- Package versions: pin package versions, remove build tools from final image
- Image scanning: confirm Trivy scan passes before pushing to registry

For each item:

- Bad example (what NOT to do) and good example (correct implementation)

Deliverable:

- Professional checklist document + annotated Dockerfile showing all 10 controls applied

DOCUMENT

Writing Prompt:

Write a 'Containers vs Virtual Machines — Security Trade-offs' briefing for your development team. Audience: Developers who have only used VMs. Length: 350–400 words.

- Include: why containers are faster to start but have less isolation, the three scenarios where you should still use a VM instead of a container, and the one container security control that matters most (non-root user — explain why with a specific attack scenario)

DAY
2

Container Vulnerability Scanning with Trivy

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Explain what container vulnerability scanning detects

2. Run Trivy against Docker images and interpret the output

3. Integrate Trivy into a GitHub Actions CI/CD pipeline

TRIVY

is a comprehensive, easy-to-use vulnerability scanner for containers, filesystems, and IaC. It detects CVEs in OS packages, language-specific libraries, and misconfigurations in a single tool.

What Trivy Scans

```
■■■
```

```
python:3.9 (debian 11.7) Total: 47 (HIGH: 8, CRITICAL: 3) ■■■
```

Step 4 — Scan public images for comparison

BASH

```
# Scan DVWA (from our Week 5 lab) trivy image vulnerables/web-dvwa # Scan a minimal image trivy image alpine:3.18 # Scan a distroless image (Google's minimal image) trivy image gcr.io/distroless/python3
```

Step 5 — Create a comparison table

For each image scanned, record: image name, total CVEs, HIGH count, CRITICAL count, base OS. Which image is the safest choice? Why?

Step 6 — Scan the filesystem

BASH

```
cd ~/labs/week11 trivy fs . # This scans requirements.txt and the Dockerfile # It finds vulnerable Python packages AND Dockerfile misconfigurations
```

■ ANALYSE THIS

Your vulnerable Dockerfile uses ubuntu:18.04. Trivy shows 400+ CVEs. Your secure Dockerfile uses python:3.11-slim. Trivy shows far fewer. Why does base image choice reduce the attack surface so dramatically? What is the principle called? (Hint: minimise what's installed)

BUILD *(2 Hours)*

BUILD BRIEF

Add container scanning to your Week 10 GitHub Actions pipeline and create a Container Security Report.

Step 1 — Update your GitHub Actions pipeline:

- Add a Docker build step (build your secure Dockerfile)
- Add a Trivy scan step using aquasecurity/trivy-action
- Configure: exit-code 1 on CRITICAL findings, ignore-unfixed, HIGH+CRITICAL only

Step 2 — Container Security Report (document):

- For each image scanned: name, Trivy output summary, CVE count by severity
- Comparison table: vulnerable-app:v1 vs secure-app:v1 — what changed and why
- Base image recommendation: which base image would you use for a production Python app and why

Step 3 — Add to the pipeline report:

- Which stage container scanning runs in and why it runs AFTER build but BEFORE deploy
- What happens to the pull request if Trivy finds a CRITICAL CVE

Format:

- Working updated security.yml + PDF report showing pipeline with container scan step

DOCUMENT

Writing Prompt:

Write a Container Image Policy for your organisation. This defines which base images are approved for production use. Audience: Development team.

- Required: Approved base images list (python:3.11-slim, node:18-alpine, etc.), prohibited images (latest tags, ubuntu:18.04, any unscanned image), scanning requirements (must pass Trivy HIGH+CRITICAL before push to registry), update schedule (rescan every 30 days even if code hasn't changed — new CVEs are published daily)
- Include one real example: an image that passed Trivy on day 1 but would fail 30 days later due to a newly published CVE

DAY
3

Infrastructure as Code (IaC) and Terraform Security

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Explain Infrastructure as Code and why it introduces security risks

2. Identify common Terraform security misconfigurations

3. Write basic Terraform code with security controls applied

INFRASTRUCTURE AS CODE (IaC)

is the practice of managing and provisioning computing infrastructure through machine-readable configuration files (code) rather than manual processes — making infrastructure repeatable, versionable, and auditable.

Infrastructure as Code — How It Works

```
■■■
```

No MFA on root	No MFA config	Root account can be brute forced	Enforce MFA via AWS Config rule
Unrestricted egress	egress all 0.0.0.0/0	Malware can call home freely	Restrict to required IPs/ports

Terraform Secure vs Insecure Examples

TERRAFORM — INSECURE S3 BUCKET

```
# INSECURE: Public S3 bucket with no encryption resource "aws_s3_bucket" "data" { bucket =
"company-data" acl = "public-read" # ANYONE can read this! # No encryption # No versioning # No access
logging }
```

TERRAFORM — SECURE S3 BUCKET

```
# SECURE: Private encrypted S3 bucket resource "aws_s3_bucket" "data" { bucket =
"company-data-${var.environment}" tags = { Environment = var.environment } # No ACL = private by
default } resource "aws_s3_bucket_versioning" "data" { bucket = aws_s3_bucket.data.id
versioning_configuration { status = "Enabled" } # Versioning enables recovery from accidental deletion
} resource "aws_s3_bucket_server_side_encryption_configuration" "data" { bucket =
aws_s3_bucket.data.id rule { apply_server_side_encryption_by_default { sse_algorithm = "aws:kms" #
Encrypted at rest } } } resource "aws_s3_bucket_public_access_block" "data" { bucket =
aws_s3_bucket.data.id block_public_acls = true block_public_policy = true ignore_public_acls = true
restrict_public_buckets = true # Blocks ALL public access — belt and suspenders }
```

WHY THIS MATTERS

The 2019 Capital One breach exposed 106 million credit card applications. The attacker exploited an SSRF vulnerability to access the AWS metadata service and steal IAM credentials. They then used those credentials to access misconfigured S3 buckets. The S3 buckets had overly permissive IAM policies — a misconfiguration that Checkov would have flagged in the Terraform code before deployment.

PRACTICE *(2 Hours)*

LAB 11.3.1 — Write Terraform Code and Identify Misconfigurations

Objective: Create vulnerable and secure Terraform configurations.

Prerequisites: Terraform installed (or use for analysis only); text editor

Step 1 — Install Terraform (optional — for syntax validation)

BASH

```
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/hashicorp-archive-keyring.gpg echo 'deb
[signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com
$(lsb_release -cs) main' | sudo tee /etc/apt/sources.list.d/hashicorp.list sudo apt update && sudo apt
install terraform -y terraform --version
```

Step 2 — Create a vulnerable Terraform configuration

BASH

```
mkdir -p ~/labs/week11/terraform cat > ~/labs/week11/terraform/main_vulnerable.tf << 'EOF' provider "aws" { region = "us-east-1" access_key = "AKIAIOSFODNN7EXAMPLE" # HARDCODED! secret_key = "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY" # HARDCODED! } resource "aws_s3_bucket" "user_data" { bucket = "techrise-user-data" acl = "public-read" # ANYONE can read! } resource "aws_security_group" "web" { ingress { from_port = 0 to_port = 65535 protocol = "tcp" cidr_blocks = ["0.0.0.0/0"] # ALL ports open to internet! } } resource "aws_db_instance" "main" { engine = "mysql" instance_class = "db.t3.micro" publicly_accessible = true # Database on internet! storage_encrypted = false # No encryption! password = "password123" # Weak password! } EOF echo 'Vulnerable Terraform created'
```

Step 3 — Create a secure Terraform configuration

BASH

```
cat > ~/labs/week11/terraform/main_secure.tf << 'EOF' provider "aws" { region = var.aws_region # Credentials from environment: AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY # No hardcoded credentials! } variable "aws_region" { default = "eu-west-1" } variable "db_password" { sensitive = true } # Marked sensitive } resource "aws_s3_bucket" "user_data" { bucket = "techrise-user-data-${random_id.suffix.hex}" } resource "aws_s3_bucket_public_access_block" "user_data" { bucket = aws_s3_bucket.user_data.id block_public_acls = true block_public_policy = true restrict_public_buckets = true } resource "aws_security_group" "web" { ingress { from_port = 443 to_port = 443 protocol = "tcp" cidr_blocks = ["0.0.0.0/0"] # Only HTTPS allowed } egress { from_port = 443 to_port = 443 protocol = "tcp" cidr_blocks = ["0.0.0.0/0"] # Only HTTPS out } } resource "aws_db_instance" "main" { engine = "mysql" instance_class = "db.t3.micro" publicly_accessible = false # Private! storage_encrypted = true # Encrypted! password = var.db_password # From variable! deletion_protection = true } EOF echo 'Secure Terraform created'
```

■ ANALYSE THIS

Count the security issues in main_vulnerable.tf. For each issue: which MITRE ATT&CK; tactic does it enable if exploited? Map the hardcoded credentials to a tactic, the public S3 bucket to a tactic, and the open security group to a tactic.

BUILD *(2 Hours)*

BUILD BRIEF

Create a Terraform Security Baseline — a set of reusable Terraform modules with security controls built in.

Create three secure Terraform modules:

- Module 1 — secure_s3: private bucket + encryption + versioning + access logging + public access block
- Module 2 — secure_sg: security group with principle of least privilege (only required ports, specific CIDRs)
- Module 3 — secure_rds: encrypted, private, deletion-protected RDS instance with rotation-ready password

For each module:

- All insecure options explicitly set to secure (don't rely on defaults)
- Comments explaining WHY each setting is required (e.g., # Prevents data exfiltration via public bucket)
- Variable definitions for values that change per environment

Security rationale document:

- For each module — which OWASP cloud risk does it address, which CVE/breach it prevents, how Checkov will verify it

Format:

- Three .tf files + one security rationale document

DOCUMENT

Writing Prompt:

Write a Cloud Security Shared Responsibility Guide for QuickPayNG. They are moving their payment app to AWS. Audience: QuickPayNG CTO who is new to cloud.

- Include: What AWS is responsible for (physical security, hypervisor, networking infrastructure), what QuickPayNG is responsible for (IAM, S3 config, encryption, OS patching), the three biggest mistakes Nigerian companies make when moving to cloud, and one real breach caused by misunderstanding the shared responsibility model (Capital One 2019)

DAY
4

IaC Scanning with Checkov and Cloud Misconfiguration Detection

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Run Checkov to scan Terraform code for misconfigurations

2. Interpret Checkov findings and map them to cloud security benchmarks

3. Add Checkov to a CI/CD pipeline to prevent misconfigured infrastructure from deploying

CHECKOV

is an open-source static analysis tool for Infrastructure as Code that finds misconfigurations in Terraform, CloudFormation, Kubernetes YAML, Helm, ARM Templates, and more — before they are deployed to cloud environments.

Checkov in the DevSecOps Pipeline

```
■■■
```

CKV_AWS_18	S3 bucket access logging enabled	aws_s3_bucket_logging: configured
CKV_AWS_52	S3 MFA delete enabled	versioning: mfa_delete = Enabled
CKV_AWS_24	Security group: no unrestricted SSH (port 22)	ingress: 0.0.0.0/0 on port 22
CKV_AWS_25	Security group: no unrestricted RDP (port 3389)	ingress: 0.0.0.0/0 on port 3389
CKV_AWS_17	RDS not publicly accessible	publicly_accessible = false
CKV_AWS_16	RDS encrypted	storage_encrypted = true
CKV_AWS_23	RDS deletion protection enabled	deletion_protection = true
CKV_AWS_28	RDS backup retention >= 7 days	backup_retention_period >= 7
CKV_AWS_3	EC2 EBS volumes encrypted	encrypted = true in ebs_block_device

Reading Checkov Output

CHECKOV OUTPUT EXAMPLE

```
Passed checks: 3, Failed checks: 8, Skipped checks: 0 Check: CKV_AWS_20: 'Ensure the S3 bucket does not have ACL defined which allows public access' FAILED for resource: aws_s3_bucket.user_data File: /terraform/main_vulnerable.tf:8-11 8 | resource "aws_s3_bucket" "user_data" { 9 | bucket = "techrise-user-data" 10| acl = "public-read" 11| } Guide: https://docs.bridgecrew.io/docs/s3_1-acl-read-permissions-everyone
```

Adding Checkov to GitHub Actions

YAML — Add to .github/workflows/security.yml

```
- name: Run Checkov IaC scan uses: bridgecrewio/checkov-action@master with: directory: terraform/ framework: terraform output_format: cli soft_fail: false # true = warn only, false = block pipeline skip_check: CKV_AWS_144 # Skip if accepted risk (document why!) # Pipeline FAILS if any misconfiguration found # Developers must fix terraform code before merging
```

WHY THIS MATTERS

A Bridgecrew report analysed 400,000 Terraform repositories on GitHub and found that 44% had at least one S3 bucket with public read access, 73% had security groups open to 0.0.0.0/0, and 91% had unencrypted RDS instances. These are the exact misconfigurations Checkov catches. Five minutes of Checkov scanning before deployment would have prevented the majority of cloud breaches in 2023.

PRACTICE *(2 Hours)*

LAB 11.4.1 — Scan Terraform with Checkov and Fix Findings

Objective: Run Checkov against vulnerable Terraform and remediate all findings.

Prerequisites: Checkov installed; Terraform files from Day 3

Step 1 — Install Checkov

BASH

```
pip3 install checkov --break-system-packages checkov --version
```

Step 2 — Scan the vulnerable Terraform

BASH

```
checkov -f ~/labs/week11/terraform/main_vulnerable.tf # Expected: Multiple FAILED checks # Note each check ID and what it means
```

Step 3 — Scan the secure Terraform

BASH

```
checkov -f ~/labs/week11/terraform/main_secure.tf # Expected: Most checks PASS # Note any remaining failures — can they be fixed?
```

Step 4 — Fix a specific Checkov failure

Take one FAILED check from the vulnerable file. Read the Checkov guidance link. Write the fix in Terraform. Re-run Checkov to confirm PASS.

Step 5 — Scan the Dockerfile with Checkov

BASH

```
cd ~/labs/week11 checkov -d . --framework dockerfile # Expected findings on vulnerable Dockerfile: # CKV_DOCKER_2: Healthcheck not found # CKV_DOCKER_3: User not specified # CKV_DOCKER_4: Don't use latest tag (use specific version)
```

LAB 11.4.2 — Complete DevSecOps Pipeline — All Security Gates

Objective: Run the complete security pipeline covering all tools from Weeks 10–11.

Prerequisites: GitHub repository from Week 10; all tools installed

Add the following to your Week 10 security.yml and push to GitHub. All 5 stages should run:

1. SAST — Bandit/Semgrep on Python code
2. Dependency scan — pip-audit on requirements.txt
3. Secrets detection — GitLeaks on repository history
4. Container scan — Trivy on Docker image
5. IaC scan — Checkov on Terraform files

Watch the pipeline run in GitHub Actions. Screenshot each stage passing or failing.

■ ANALYSE THIS

Your complete pipeline now has 5 security gates. Which gate runs fastest? Which takes longest? If a developer commits code at 9:00 AM and the pipeline takes 4 minutes total, by 9:04 AM they know if their code has any security issue from code to infrastructure. How does this compare to a traditional security review process that might take days or weeks?

BUILD *(2 Hours)*

BUILD BRIEF

Create a Complete IaC Security Assessment for your Terraform configurations.

Required sections:

- Checkov scan results: full output for both main_vulnerable.tf and main_secure.tf
- Findings table: Check ID, Resource, Severity, What It Means, Business Risk, Fix Applied
- Before/After comparison: for each fixed finding — the vulnerable line vs the secure line with explanation
- Cloud Security Benchmark mapping: map your Checkov findings to CIS AWS Benchmark controls
- Pipeline evidence: screenshot of your complete 5-stage GitHub Actions pipeline running
- False positive analysis: are there any Checkov findings you would mark as accepted risk? Document the business justification

Format:

- Professional security assessment report — same standard as Week 9 pentest report

DOCUMENT

Writing Prompt:

Write a Cloud Misconfiguration Risk Brief for QuickPayNG's board. They are planning to store customer payment data in AWS S3. Audience: Board of Directors (non-technical). Length: 400–500 words.

- Include: What a misconfigured S3 bucket means in plain English (anyone on the internet can access your files), what data would be at risk (50,000 customer payment records), regulatory implications (NDPA 2023 breach notification, CBN data requirements, potential PCI-DSS implications), how Checkov prevents this (automated scan blocks misconfigured infrastructure before it deploys), cost comparison: Checkov implementation ($0) vs breach cost ($millions)

DAY
5

Review, Assessment & Portfolio

LEARN *(1 Hour) — Week Review*

LEARNING OBJECTIVES

1. Demonstrate all Week 11 skills without referencing notes
2. Complete the Mini Project 3 submission (due today)
3. Begin Week 12 Capstone preparation

Week 11 Mind Map — Container & IaC Security



PRACTICE *(2 Hours) — Skills Assessment Labs*

LAB 11.5.1 — Container Security Rapid Challenge

Objective: Complete container security tasks from memory — no notes, 30 minutes.

Prerequisites: Docker; Trivy; Bandit; no notes

Complete all tasks from memory:

1. Write a Dockerfile from memory that: uses python:3.11-slim, creates a non-root user, copies app.py, runs as non-root
2. Build the image and run Trivy against it — what is the CVE count?
3. Run docker inspect on the image — confirm no secrets are visible in Env section
4. Write the GitHub Actions YAML step that runs Trivy and fails on CRITICAL CVEs
5. List 3 Checkov check IDs from memory and what each enforces

LAB 11.5.2 — Terraform Misconfiguration Hunt

Objective: Find all security issues in a provided Terraform file — no notes, 20 minutes.

Prerequisites: Checkov installed; no notes

Run Checkov against main_vulnerable.tf and identify all failures. For each failure: what is the misconfiguration, what attack does it enable, what is the one-line Terraform fix.

BUILD *(2 Hours) — Mini Project 3 Final Submission*

■ MINI PROJECT 3 — Complete DevSecOps Pipeline — FINAL SUBMISSION

This is your Mini Project 3 final submission. It demonstrates a complete DevSecOps pipeline from code security to container security to infrastructure security — the full Phase 3 skill set.

Deliverables:

- GitHub repository: techrise-payment-api with README explaining the project
- Application code: Python Flask API with vulnerable branch and secure branch
- Complete CI/CD pipeline (.github/workflows/security.yml) with 5 security gates: SAST, dependency scan, secrets detection, Trivy container scan, Checkov IaC scan
- Vulnerable branch results: run all 5 tools and document all findings in a professional table
- Secure branch results: pipeline passes all 5 gates — screenshot of all green checks
- Secure Dockerfile: uses python:3.11-slim, non-root user, no baked secrets
- Terraform configuration: secure S3, security group, and RDS with Checkov passing
- Container Security Report: Trivy scan comparison (vulnerable vs secure image)
- IaC Security Report: Checkov findings and remediations
- AI Integration Log: prompts used for YAML generation, what was corrected
- DevSecOps Maturity Assessment: 1-page — where your pipeline sits on a maturity scale and what would Level 5 look like

Tools: Git · GitHub Actions · Bandit · Semgrep · pip-audit · GitLeaks · Trivy · Checkov · Python Flask · Terraform · Claude AI (documented)

Mini Project 3 — Final Submission Details

- Due: End of Week 11 (today)
- Format: GitHub repository link + PDF report (minimum 20 pages)
- Evaluation: Working pipeline with all 5 gates (40%), Findings quality (25%), Secure code/IaC (20%), Documentation (15%)
- Week 12 Capstone builds directly on this project — keep your repository clean and documented

DOCUMENT *(1 Hour) — Week Summary*

DOCUMENT

Writing Prompt:

Write your Week 11 learning reflection. Length: 300–400 words.

- The most important container security control and why it matters more than all the others (non-root user — or make the case for a different one)
- How does IaC security scanning (Checkov) compare to traditional security audits? What does it catch that a manual audit misses?
- Your complete 5-stage pipeline now covers: code, dependencies, secrets, containers, infrastructure. What is still NOT covered? (Hint: runtime security, DAST, monitoring)
- If you were hired as a DevSecOps engineer at QuickPayNG tomorrow, which of the 5 pipeline stages would you implement first and why?

Weekly Checklist (Student-Facing)

- ■ I can explain the difference between a VM and a container and the security implications of each
- ■ I have built a vulnerable Docker image and a secure Docker image showing the differences
- ■ I have run Trivy against a Docker image and interpreted the CVE output
- ■ I have integrated Trivy into a GitHub Actions pipeline with exit-code 1 on CRITICAL
- ■ I have written Terraform code with at least one security misconfiguration and then fixed it
- ■ I have run Checkov against Terraform and identified all failing checks
- ■ My GitHub Actions pipeline now has all 5 security gates (SAST, deps, secrets, Trivy, Checkov)
- ■ I can name 5 Checkov check IDs and what each enforces from memory
- ■ My Mini Project 3 is complete and submitted
- ■ I understand the Cloud Shared Responsibility Model and can explain what AWS vs customer secures

Skills Verification

Skill	How to Demonstrate It	Pass Standard
Container vs VM	Explain security difference with one example each	Correct explanation; isolation difference identified
Secure Dockerfile	Show Dockerfile with non-root user and no baked secrets	USER instruction present; no ENV with credentials
Trivy scan	Run Trivy on an image and show output	CVE count shown; severity breakdown visible
Trivy CI integration	Show GitHub Actions step with Trivy scan	YAML correct; exit-code 1 on CRITICAL
Terraform misconfiguration	Identify 3 issues in main_vulnerable.tf	All 3 issues found with correct Checkov check ID
Checkov scan	Run Checkov on Terraform and show FAILED checks	Multiple failures shown; at least one fixed
Complete pipeline	Show GitHub Actions with all 5 security gates	All 5 stages visible in Actions tab

Self-Assessment Quiz

Instructions

- No notes — answer from memory.
- Write on separate sheet.
- Key in Instructor Notes.

Concept Questions

1. Explain why running a container as root is dangerous. Give a specific attack scenario where a non-root container would contain a breach, but a root container would allow full host takeover.
2. What is the difference between a Docker image and a Docker container? If a container has a vulnerability, does deleting the container fix it?
3. What does 'publicly_accessible = true' on an AWS RDS instance mean in practical terms? What could an attacker do with access to a publicly accessible MySQL database?

Tool Questions

4. You run Trivy against your Docker image and get: CRITICAL: 2, HIGH: 15, MEDIUM: 47. Your pipeline is configured with --severity HIGH,CRITICAL --exit-code 1. Does the pipeline pass or fail? What must you do to make it pass?
5. Checkov check CKV_AWS_20 fails on your Terraform. The guidance says: 'Ensure the S3 bucket does not have ACL defined which allows public access.' Write the Terraform fix (the exact resource block with correct setting).

Scenario Questions

6. A developer says: 'I use a private Docker Hub repository so my images are safe — no need to scan them.' Give two specific reasons why this is incorrect.
7. Your team is deploying a Terraform configuration with these settings: publicly_accessible = true on RDS, cidr_blocks = 0.0.0.0/0 on all security groups, S3 bucket with acl = public-read-write. Map each misconfiguration to a MITRE ATT&CK; tactic and explain the attack chain.

Integration Questions

8. Describe the complete 5-stage security pipeline you built in Weeks 10–11. For each stage: tool name, what it checks, and what happens when it fails. Draw this as a text diagram.

Additional Resources

Container Security

- Docker Security Best Practices: https://docs.docker.com/develop/security-best-practices/
- CIS Docker Benchmark: https://www.cisecurity.org/benchmark/docker
- Trivy Documentation: https://aquasecurity.github.io/trivy/
- Distroless Images (Google): https://github.com/GoogleContainerTools/distroless

IaC Security

- Checkov Documentation: https://www.checkov.io/
- Terraform Security Best Practices: https://developer.hashicorp.com/terraform/tutorials/configuration-language/sensitive-variables
- tfsec (alternative to Checkov): https://github.com/aquasecurity/tfsec
- CIS AWS Benchmark: https://www.cisecurity.org/benchmark/amazon_web_services

Cloud Security

- AWS Security Hub: https://aws.amazon.com/security-hub/
- OWASP Cloud Top 10: https://owasp.org/www-project-cloud-native-application-security-top-10/
- AWS Well-Architected Framework (Security Pillar): https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/welcome.html

Free Practice

- TryHackMe — Container Security: https://tryhackme.com/room/containersecurity
- KodeKloud — Terraform for beginners (free tier): https://kodekloud.com
- AWS Free Tier (for Terraform practice): https://aws.amazon.com/free/

Looking Ahead — Week 12

Preview — Week 12: Capstone Project

- Week 12 is the final week — your Capstone Project brings together all 11 weeks
- You will demonstrate a complete security engagement: from reconnaissance (Week 4) through detection (Week 6) to DevSecOps (Weeks 10–11)
- The Capstone requires a professional report (Week 9 skills) covering all phases
- Your Week 11 pipeline becomes part of the Capstone deliverable
- Presentations to the cohort — 10 minutes per student/group

What to Prepare Before Week 12

- Ensure your Mini Project 3 pipeline is working and documented
- Review your best work from all 11 weeks — you will select highlights for the Capstone report
- Review the MITRE ATT&CK; framework (Week 9) — the Capstone requires mapping all activities
- Prepare a 10-minute presentation structure: 3 minutes attack, 3 minutes detection, 3 minutes DevSecOps, 1 minute lessons learned

Capstone Scope (Preview)

- Phase 1 — Red Team (Week 12 Day 1–2): demonstrate reconnaissance and exploitation in your lab
- Phase 2 — Blue Team (Week 12 Day 3): show detection in Splunk/Snort and forensic evidence
- Phase 3 — DevSecOps (Week 12 Day 4): demonstrate your complete pipeline blocking a vulnerability
- Final Report: professional security engagement report covering all three phases
- Presentation: 10 minutes to cohort panel

Phase 3 Complete After Week 11

- By completing Week 11 you have finished Phase 3: DevSecOps and Cloud Security
- Skills gained: CI/CD pipeline security, SAST, dependency scanning, secrets detection, container scanning, IaC security scanning
- Week 12 is the grand finale — bring your best work